

Cahier des Besoins

0.1 Création du modèle User dans la base de données et le serveur

Description :

- Définir la structure du modèle User (identifiant, informations personnelles, mot de passe, métadonnées, etc.).
- Générer et configurer la table correspondante dans la base de données.

Gestion d'erreurs :

- 500 Internal Server Error : Erreur lors de la création du modèle ou de la table en base de données.

Tests :

- Vérifier que la table User est bien créée en base.
- Vérifier que les champs obligatoires sont présents et correctement typés.

0.2 Mise en place de l'inscription et de l'authentification

Description :

- Développer les contrôleurs nécessaires pour la création de compte (inscription) et la connexion (authentification).
- Implémenter les services associés pour gérer la logique métier (validation, interaction avec la base de données).

Gestion d'erreurs :

- 200 OK : Authentification réussie.
- 201 Created : Compte créé avec succès.
- 400 Bad Request : Données invalides ou manquantes.
- 401 Unauthorized : Identifiants incorrects.
- 409 Conflict : Utilisateur déjà existant.

Tests :

- Vérifier qu'un utilisateur peut s'inscrire avec des données valides.
- Vérifier qu'un utilisateur existant ne peut pas être recréé.
- Vérifier qu'un utilisateur peut se connecter avec les bons identifiants.
- Vérifier qu'un mauvais mot de passe est refusé.

0.3 Sécurisation des mots de passe

Description :

- Implémenter un système de hachage sécurisé (BCrypt) pour le stockage des mots de passe.
- Garantir qu’aucun mot de passe ne soit stocké ou manipulé en clair dans la base de données.

Gestion d’erreurs :

- 500 Internal Server Error : Erreur lors du hachage ou du stockage du mot de passe.

Tests :

- Vérifier que le mot de passe est stocké sous forme hashée.
- Vérifier qu’aucun mot de passe en clair n’est présent en base.
- Vérifier que la comparaison fonctionne lors de l’authentification.

0.4 Association entre un utilisateur et ses images

Description :

- Mettre en place une relation stricte garantissant qu’une image appartient à un seul utilisateur.
- Ajouter les clés étrangères nécessaires dans la base de données.
- S’assurer que toute opération sur une image soit effectuée dans le contexte de l’utilisateur propriétaire.
- Ajout d’images par l’utilisateur sur son espace personnel.

Gestion d’erreurs :

- 403 Forbidden : Accès à une image ne appartenant pas à l’utilisateur.
- 404 Not Found : Image inexistante.
- 500 Internal Server Error : Problème de liaison en base de données.

Tests :

- Vérifier qu’une image est correctement liée à un utilisateur.
- Vérifier qu’un utilisateur ne peut pas accéder aux images d’un autre.
- Vérifier l’intégrité des clés étrangères.

0.5 Gestion de la suppression des images par l’utilisateur

Description :

- Permettre à un utilisateur authentifié de supprimer l’une de ses images.
- Garantir que seule l’image appartenant à l’utilisateur puisse être supprimée par celui-ci.

Gestion d’erreurs :

- 204 No Content : Image supprimée avec succès.

- 403 Forbidden : Tentative de suppression d’une image appartenant à un autre utilisateur.
- 404 Not Found : Image inexistante.
- 401 Unauthorized : Utilisateur non authentifié.

Tests :

- Vérifier qu’un utilisateur peut supprimer sa propre image.
- Vérifier qu’il ne peut pas supprimer celle d’un autre.
- Vérifier le comportement avec une image inexistante.

0.6 Intégration Front-End

Description :

- Les pages Connexion et Inscription ont été développées côté Front-End.
- La communication avec le serveur (API Back-End) est opérationnelle pour :
 - l’envoi des données d’inscription,
 - l’authentification de l’utilisateur,
 - la gestion des retours d’erreur et des états de connexion.
- Le Front-End est connecté au serveur via des appels API sécurisés (Axios).

Gestion d’erreurs :

- 400 Bad Request : Données invalides envoyées par le client.
- 401 Unauthorized : Échec d’authentification.
- 500 Internal Server Error : Erreur côté serveur.

Tests :

- Vérifier que les appels API fonctionnent correctement depuis le Front-End.
- Vérifier l’affichage des erreurs côté utilisateur.
- Vérifier la gestion de session (connexion/déconnexion).

0.7 Publication d’images par l’utilisateur

Description : Mettre en place la fonctionnalité permettant à un utilisateur connecté de publier une image afin qu’elle soit visible publiquement par les autres utilisateurs.

Gestion d’erreurs :

- 200 OK : Publication réussie.
- 401 Unauthorized : Utilisateur non authentifié.
- 404 Not Found : Image inexistante.
- 500 Internal Server Error : Erreur lors de la publication.

Tests :

- Vérifier qu’un utilisateur connecté peut publier une image.
- Vérifier qu’une image publiée est visible par les autres utilisateurs.

0.8 Interactions des autres utilisateurs sur les images publiées

Description : Développer les mécanismes d'interaction sur les images (likes, commentaires, etc.).

Gestion d'erreurs :

- 200 OK : Interaction enregistrée.
- 401 Unauthorized : Utilisateur non authentifié.
- 404 Not Found : Image inexistante.
- 400 Bad Request : Données invalides.

Tests :

- Vérifier l'ajout d'un like.
- Vérifier l'ajout d'un commentaire.
- Vérifier que les interactions sont bien associées à l'image.

0.9 Amélioration du Front-End

Description : Optimiser l'interface afin de la rendre plus fluide, intuitive et accessible.

Gestion d'erreurs :

- 500 Internal Server Error : Erreurs lors des appels API.

Tests :

- Vérifier la fluidité de navigation.
- Vérifier le bon affichage sur différents écrans.
- Tester les interactions utilisateur principales.